Assignment 4: Lighting (Medium)

10 Points Possible

3/1/2026

Attempt 1

In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

3/11/2026

Details

Videos:

- Prof. James' Tutorials: **YouTube Playlist (https://youtube.com/playlist?list=PLbyTU_tFIkcNbxNKJWEcjfBp97fKE2UuW)**
- Lab Section (Fall 2021): **Full video (week 7) (https://media.ucsc.edu/V/Video?v=3891361&node=13303029&a=239173022&autoplay=1)** || **Full video (week 8) (https://media.ucsc.edu/V/Video?v=3919112&node=13370855&a=306271846&autoplay=1)** || **YouTube Playlist (https://youtube.com/playlist?list=PLbyTU_tFIkcMH-g6Bq9eBGtJTIFbvx0He)**
- Lab Section (Fall 2020): **YouTube Playlist (https://youtube.com/playlist?list=PLbyTU_tFIkcNw0pb6LTx1jL8TvOqs6Bmr)**

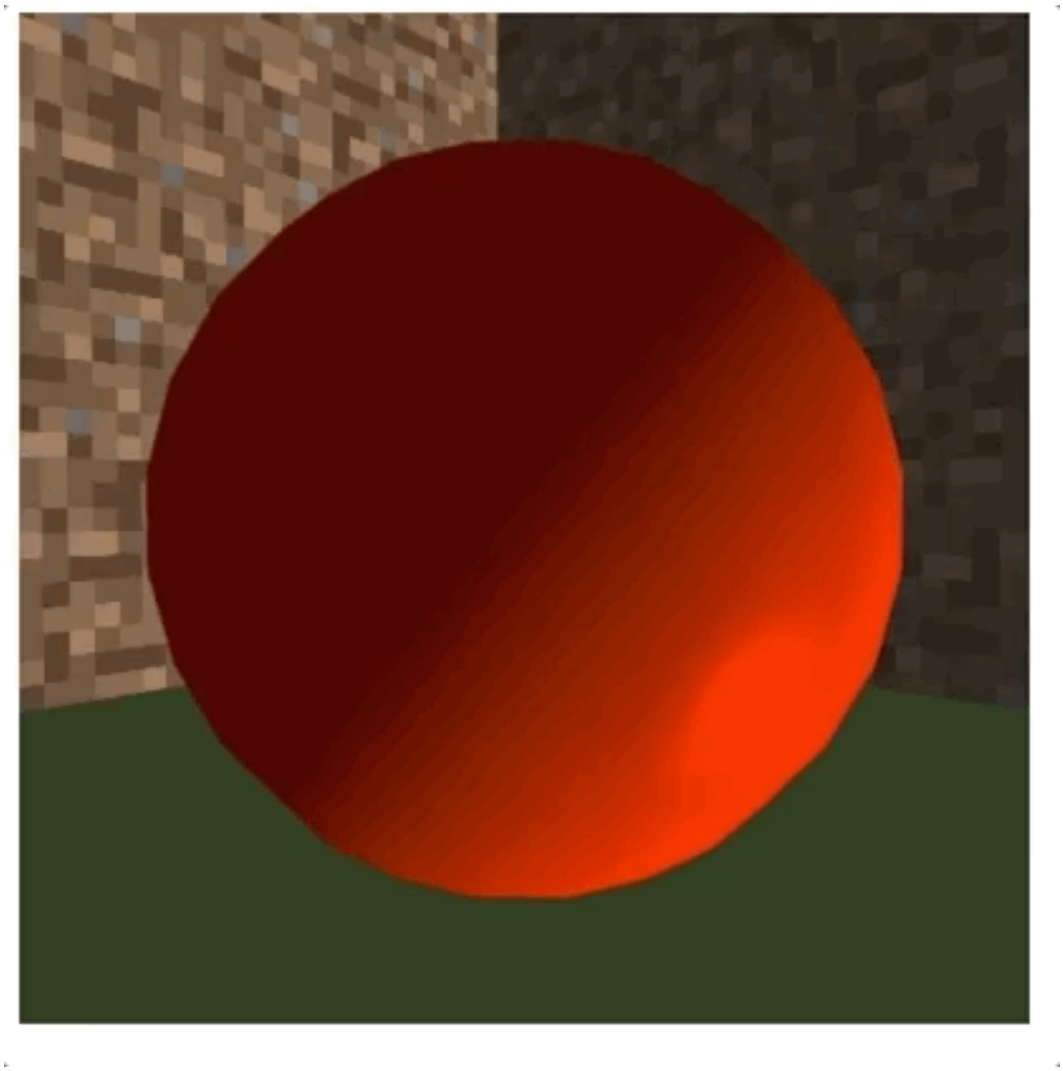
This video may display YouTube ads.

Continue to YouTube content

Objectives:

To have a light that illuminates your objects.

Introduction:



The assignment requires a cube and a couple spheres, and a single **point** light that will illuminate the objects of your world using the **Phong Shader**. Your application is required to have the following features:

- A Cube - The most basic object that you should be able to create by now.
- A sphere in your world. Why? Its easier to tell if lighting is working correctly on spheres.
- A point light that changes location with a slider, possibly spinning around your world, or just moving back and forth.
- A way to tell where the light is. Easiest is to just render a cube at the light location.
- Some method of moving the camera around the objects, either rotation slider or keyboard
- Each object (ground, walls, animal, and spheres) of the world is illuminated using a Phong Shader
  - Phong Shader models a combination of ambient, diffuse and specular lighting.
  - Specular and Diffuse lighting calculations require the normal of the vertices. So you will have to calculate the normal of each vertex and pass this as input to the shader.
- A button to turn lighting on/off
- A button to turn normal visualization on/off
- Additional requirements that do not have helper videos
  - Implement a spot light which focuses light in a particular direction (in addition to the first light you had), be able to turn on/off the lights individually.
    - You may find this link helpful, section 7.2.6

    - **http://math.hws.edu/graphicsbook/c7/s2.html (http://math.hws.edu/graphicsbook/c7/s2.html)**
  - Load an OBJ with complex shape and see the lighting on this
  - Your lighting is working either with your animal or with your world from A2 or A3

Instructions:

1. This description assumes you are starting from your prior code. Choose either your animal or your virtual world. However its allowable to start over from scratch. If so then just get a single cube on the screen, and make sure you can change the camera angle somehow.
2. We need to pass normals to the shaders.
   - In the same way that we previously created a buffer to pass UV coordinates for texture, we now need another buffer to pass a Normal for each vertex. Create a new attribute variable in your vertex shader, and modify your javascript code to create and bind this buffer.
   - Add a button to turn Normal Visualization on/off. It should toggle you between your actual color/texture and just looking at what is in the Normal attribute. Use a uniform variable to pass this status to your shader and just use an 'if' statement in the shader to decide which way to color the object.
   - Change your object color in GL from whatever it was to instead set gl_FragColor = Normal. That is, we just take the value in Normal and stick it in the color. We haven't calculated Normals yet, so just set all the Normals to (1.0,1.0,0.0) in javascript for now so we can debug this point and your objects should all be yellow.
   - Now its time to actually pass some real normals. For your cube object, you will need to define the vertex Normals (similar to how you defined the vertex UV coordinates in the last assignment). For cubes you can just draw a picture on paper and figure it out and type the normals in. Alternately you could write a function that looks at the triangle and uses 'edge1 cross edge2' to find the normal or the triangle. Either way, calculate some normals, and check that they are coming through to your shaders as expected using your Normal Visualization.
3. Now lets add a sphere.
   - I suppose it would be possible to calculate the vertices by hand and hard code them, similar to the cube, but its likely more convenient to write a loop, and use some trig to calculate the vertex locations. Similar to calculating a circle in the first assignment. Add the vertex locations to the attribute buffer. Check that you see a sphere on screen (it will just look like a circle since there is no lighting yet).
   - Add normals to your sphere. Note that spheres are convenient objects in that if they are centered on the origin, then Normal = Position for each vertex, so you can just copy the prior buffer. Check that these normals are coming through with Normal Visualization. Google 'sphere normals' and look at images to make sure yours look sort of like what is expected.
4. Now we are ready to add a light. Make a global variable in javascript for the light position, and a new uniform variable to pass this information to your shaders. Update your render() function to pass the current light position. Also render a little cube at the light location.
5. Animate your light: In your animation tick() function update the javascript global light position so that it moves around your world. Also, add a slider to move the light position around your world.
6. Now we need to add lighting

- Start with the most basic lighting to make sure we are passing the light into the shaders correctly. In your vertex shader calculate 'normalize(Light) dot normalize(Normal)' and assign this to a varying variable v_Lighting. In your fragment shader modify your pixel color as gl_FragColor = v_Lighting * (whatever it was before). You should now see something that looks like lighting variation that seems to rotate around but is sort of weird if your object has been rotated. It will be right on objects that were never rotated.
- So far our light is probably in world coordinates and our normals are in object coordinates. We need to make the coordinates match. If you are calculating glPosition = u_projMatrix * u_viewMatrix * u_modelMatrix * a_position, then your positions are undergoing transforms to get to screen space. We want to get our lighting and normals both into 'World Coordinates'. You will need to find your vertex in world coordinates as worldPos = u_modelMatrix * a _position (ignoring camera and projection). The proper light vector at this vertex is then lightVector = u_LightPos-worldPos. The normal also needs to transform into world coordinates, so you will need a 'normal matrix' to transform the normal. You could calculate the normalMatrix either in javascript or GLSL. Calculate these, and update your (N dot L) calculation to use these proper values with everything in world coordinates.
- Move our lighting calculation to the fragment shader. You previously calculated N dot L in the vertex shader and just passed v_Lighting to the fragment shader. Instead pass v_NormalDir and v_LightDir to the fragment shader, and calculate lighting in the fragment shader.
- Now we have all the data needed passed all the way to the fragment shader. Modify the fragment shader to calculate Phong shading - ambient + diffuse + specular. You can just hard code values for ambient and specular coefficient. The only things that actually will be getting passed from javascript are the diffuse colors. That is the texture or color values that you were previously using. Everything else is calculated in the shader.
- Finally, add a slider to change the color of your light.

7. Add a button to turn lighting on and off. Keep a javascript global variable for this status, create a uniform variable to pass the information to GLSL, and update your render() function to pass the current state. Modify your fragment shader to use lighting or not depending on status. Verify that you can turn lighting on/off.
8. Thats the end of the help and hints videos!
9. Get your world or animal working together with lighting.
10. Get a spotlight working.
11. Get an .obj model loaded in your scene and get lighting working on it. You may create a `Model.js` class similar to the one you implemented in the lab, or use another method of your choice to load and render the model. If you choose to parse `.obj` files the same way as in the lab, we've provided some sample models under Files -> OBJ Models. The `model_info.pdf` file in that folder includes preview images of each model along with their vertex and face counts. If you would like to use `.obj` files from other sources, keep in mind that not all models will be compatible with the lab's loader by default. To make them compatible, follow the conversion instructions provided in **this pdf. (https://canvas.ucsc.edu/courses/88811/files/12181191?wrap=1) (https://canvas.ucsc.edu/courses/88811/files/12181191/download?download_frd=1)**

Resources

- Readings:
  - (WebGL) Matsuda/Lea Ch8, (Chp 6 and Appendix B are GLSL reference)
  - (WebGL) The normal matrix: **http://www.lighthouse3d.com/tutorials/glsl-12-tutorial/the-normal-matrix/ (http://www.lighthouse3d.com/tutorials/glsl-12-tutorial/the-normal-matrix/)**
  - Generating a sphere mesh: **https://medium.com/game-dev-daily/four-ways-to-create-a-mesh-for-a-sphere-d7956b825db4 (https://medium.com/game-dev-daily/four-ways-to-create-a-mesh-for-a-sphere-d7956b825db4)**
  - (WebGL) Phong shading: **http://www.mathematik.uni-marburg.de/~thormae/lectures/graphics1/code/WebGLShaderLightMat/ShaderLightMat.html (http://www.mathematik.uni-marburg.de/~thormae/lectures/graphics1/code/WebGLShaderLightMat/ShaderLightMat.html)**
  - (WebGL) Phong shading with color change: **http://www.cs.toronto.edu/~jacobson/phong-demo/ (http://www.cs.toronto.edu/~jacobson/phong-demo/)**

What to Turn in:

(%24CANVAS_COURSE_REFERENCE%24/file_ref/gaa68d350951bf96b83681a987bb395f3/download)

1. **Canvas Submission**

Zip your entire project and submit it to Canvas under the appropriate assignment. Name your zip file "[FirstName]_[LastName]_Assignment_0.zip" (e.g. "Lucas_Ferreira_Assignment_0.zip").

2. **Live Hosted Submission**

You will upload your submission to GitHub Pages (or any other service of your choosing). If you use GitHub Pages, **click here (https://canvas.ucsc.edu/courses/88811/pages/submission-guide?module_item_id=1869747)** to learn how to set it up.

WHEN SUBMITTING YOUR PROJECT ON CANVAS, PLACE YOUR SITE LINK AS A COMMENT OF THE SUBMISSION.

Read the **Submission Guide (https://canvas.ucsc.edu/courses/88811/pages/submission-guide?module_item_id=1869747)** for further explanation on how to submit your assignment.

View Rubric

ASG4 Rubric.

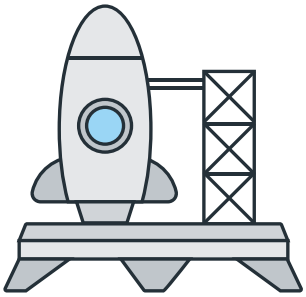
Criteria	Ratings		Points
Created a sphere.	**Full Marks** **1 pts**	**No Marks** **0 pts**	/1 pts

Criteria	Ratings			Points
Lighting (ambient+diffuse+specular) works correctly and the lighting color changes correctly with slider.	Full Marks 2.5 pts	Lighting correct, no color added 1.5 pts	No Marks 0 pts	/2.5 pts
A visual marker of light location exists.	Full Marks 0.5 pts		No Marks 0 pts	/0.5 pts
A user interface button to turn on and off lighting	Full Marks 1 pts		No Marks 0 pts	/1 pts
The point light moves around the world over time and also using the slider.	Full Marks 1 pts		No Marks 0 pts	/1 pts
A spot light is added.	Full Marks 1 pts		No Marks 0 pts	/1 pts
Button to visualize normals with color	Full Marks 1 pts		No Marks 0 pts	/1 pts
OBJ Loaded view longer description	Yes 1 pts		No 0 pts	/1 pts
Animal or World is integrated with this assignment	Yes 1 pts		No 0 pts	/1 pts

Choose a submission type

Upload

More



Choose a file to upload

or

Webcam Photo

Canvas Files

Submit Assignment